

Parallel Thinking: When More Workers Help, and When They Don't

Programming and Data Structures — Week 14, Lecture 1

Dr. Innocent Nyalala

Objectives

By the end of this lecture, you will be able to explain the difference between a problem that parallelizes naturally and one that fundamentally resists parallelization, using algorithms already covered in this course as concrete examples; identify which specific algorithms from earlier in this course parallelize easily, and explain precisely why; state Amdahl's Law and use it to compute the actual, bounded speedup limit for a workload that is only partially parallelizable; and implement a simple parallel map operation using Python's `multiprocessing` module, measuring the real, actual speedup achieved rather than assuming it.

Outline

Today covers a kitchen-with-many-cooks analogy for parallel work; naturally parallel problems, identified directly from algorithms already covered in this course; fundamentally sequential problems, and precisely why some computations resist being split across workers at all; Amdahl's Law, derived and applied to a concrete partially-parallel workload; a real parallel map operation, implemented and measured, not merely described; why measured speedup always falls short of the naive "N workers means N times faster" assumption; the MTech-track material on parallel merge sort and precisely where its speedup comes from; an in-class quiz; and a summary.

The kitchen-with-many-cooks analogy

Picture chopping 100 separate vegetables for a large meal: hiring 4 cooks genuinely helps here, since each can take roughly 25 vegetables and work entirely independently, finishing in roughly a quarter of the time one cook alone would need. Now picture baking a single cake: hiring 4 cooks does not help proportionally at all, since mixing the batter, then baking it, then letting it cool must happen in that specific sequential order, regardless of how many cooks are standing by. The difference is not

about how many workers are available — it is about whether the task itself can genuinely be split into independent pieces that do not depend on each other's results.

Naturally parallel problems, from this course's own algorithms

Several algorithms already covered in this course parallelize naturally, because their subproblems are genuinely independent. Week 6 Lecture 1's merge sort recursively sorts a left half and a right half; these two sorts share no data and neither depends on the other's result to even begin, making them naturally assignable to two separate workers. Applying a function to every element of an array — a “map” operation — is naturally parallel for the identical reason: each element's computation does not depend on any other element's. Matrix multiplication is naturally parallel too: each cell of the output matrix depends only on one specific row of one input and one specific column of the other, computable entirely independently of every other output cell.

Fundamentally sequential problems: why some cannot be split

Some computations fundamentally resist being split into independent pieces, because each step genuinely requires the previous step's completed result. Bellman-Ford's relaxation rounds, covered earlier in this course, are fundamentally sequential in this sense: round $k + 1$'s correctness depends on round k 's results already being finalized, with no way to skip ahead or compute rounds out of order. Traversing Week 4's linked list is similarly sequential: reaching node k requires having already followed the chain through nodes 1 through $k - 1$, since there is no direct-address formula the way an array provides — there is simply no way to “jump ahead” to parallelize this particular traversal.

Amdahl's Law, derived

Amdahl's Law formalizes exactly the kitchen-and-cake distinction. Let p denote the fraction of a program's total work that can genuinely be parallelized, leaving $(1 - p)$ as the fundamentally sequential remainder. With N workers, the parallel portion's time shrinks to p/N of its original duration, but the sequential portion still requires the full $(1 - p)$ regardless of how many workers are available. The total time with N workers, relative to the original single-worker time of 1, is therefore $(1 - p) + p/N$, giving a speedup of $\frac{1}{(1 - p) + p/N}$ — a precise formula, not merely an intuition.

Applying Amdahl's Law to a concrete workload

A program's work is 90% parallelizable ($p=0.9$), 10% fundamentally sequential. Using 8 workers ($N=8$):

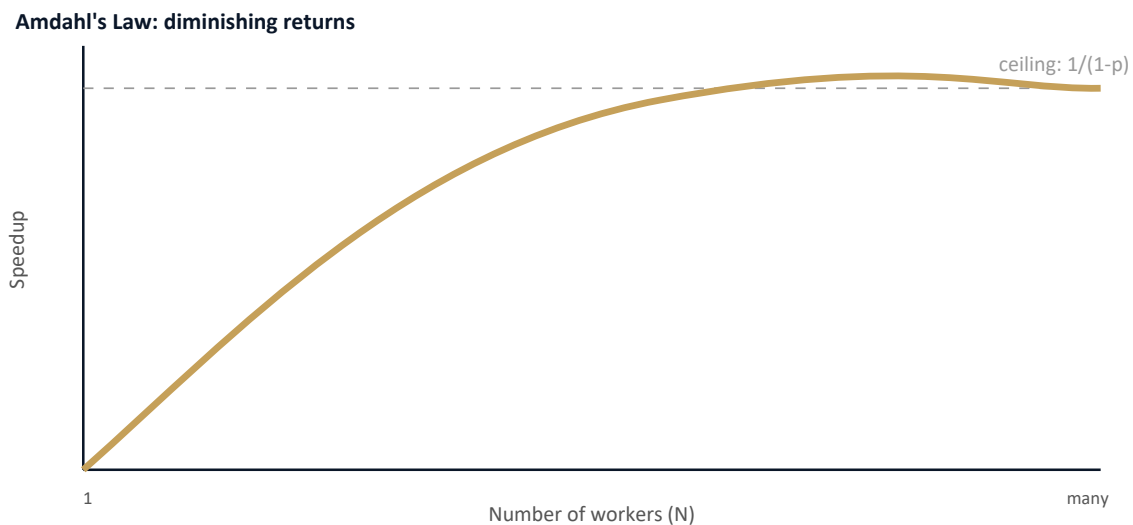
$$\text{speedup} = 1 / ((1-0.9) + 0.9/8) = 1 / (0.1 + 0.1125) = 1 / 0.2125 \quad 4.7\times$$

Using UNLIMITED workers ($N \rightarrow \text{infinity}$):

$$\text{speedup} = 1 / (1-p) = 1 / 0.1 = 10\times \quad \text{-- a hard ceiling, no matter how many workers exist}$$

Applying the formula to a concrete workload that is 90% parallelizable: with 8 workers, the speedup is $1/((1 - 0.9) + 0.9/8) = 1/0.2125 \approx 4.7\times$ — noticeably less than the naive “8 workers, 8 times faster” assumption. Taking the number of workers to infinity reveals a hard ceiling: speedup approaches $1/(1 - p) = 1/0.1 = 10\times$ and can never exceed this, no matter how many workers are ever added, since the sequential 10% remains fixed regardless of parallel resources.

The picture: Amdahl's Law's diminishing returns



The curve rises quickly with the first few added workers, then flattens dramatically, approaching but never reaching the horizontal asymptote at $1/(1 - p)$ — this is the visual signature of Amdahl's Law's diminishing returns, worth recognizing whenever evaluating whether adding more parallel workers to a real system is actually worthwhile.

Verifying the 90%-parallel numbers directly

```
def amdahl_speedup(p, n):
    return 1 / ((1 - p) + p / n)
```

```
print(amdahl_speedup(0.9, 8))          # 4.705882...
print(amdahl_speedup(0.9, float('inf')) # exactly 10.0, the hard ceiling
```

Computing this lecture’s 90%-parallelizable example directly confirms the hand-derived numbers: with 8 workers, the speedup is approximately 4.706, and taking the worker count to infinity converges exactly to 10.0 — the hard ceiling $1/(1 - p) = 1/0.1$. Running this small computation directly, rather than trusting the algebra alone, is worth doing whenever a formula this consequential is first encountered.

A real parallel map, implemented and measured

```
from multiprocessing import Pool
import time

def expensive_computation(x):
    return sum(i * i for i in range(x))

data = [1_000_000] * 8

start = time.perf_counter()
results_serial = [expensive_computation(x) for x in data]
serial_time = time.perf_counter() - start

start = time.perf_counter()
with Pool(4) as pool:
    results_parallel = pool.map(expensive_computation, data)
parallel_time = time.perf_counter() - start

print(f"serial: {serial_time:.2f}s    parallel (4 workers): {parallel_time:.2f}s")
```

`Pool(4).map(...)` distributes these 8 genuinely independent computations across 4 separate worker processes — exactly this lecture’s “chopping vegetables” case, since each computation depends on none of the others. Measuring both the serial and parallel running times directly, rather than assuming a clean 4x speedup, is precisely Week 2’s measurement discipline, applied here to a genuinely new setting: real speedup is always somewhat less than the number of workers, due to process-creation overhead and the results still needing to be collected back together at the end.

MTech addition — parallel merge sort, and where its speedup comes from

Revisiting Week 6 Lecture 1’s merge sort recursion tree through this lecture’s lens: the two recursive calls at every level are genuinely independent of each other and can be assigned to separate workers,

shrinking each level's wall-clock time toward $O(n/N)$ for N available workers, up until the recursion's branching factor is exhausted. The final merge step combining the two top-level results back at the root, however, remains inherently sequential — this is precisely Amdahl's Law's $(1 - p)$ term, now concretely identified within a specific algorithm this course already built from scratch, rather than left as an abstract fraction.

Why measured speedup is always less than the naive expectation

Beyond Amdahl's Law's clean mathematical ceiling, real measured speedup is further reduced by genuine implementation overhead the formula does not account for: creating worker processes costs real time, paid before any actual computation begins, and sending data to workers and collecting results back — serialization overhead — also costs real time. For sufficiently small or cheap computations, this overhead can exceed the parallel savings entirely, meaning parallelizing a genuinely cheap computation can make it measurably *slower*, not faster — a real, commonly encountered pitfall worth explicitly guarding against by measuring, never assuming parallelization always helps.

Exercise

As an exercise, run this lecture's `expensive_computation` example yourself, measuring the actual serial and parallel running times directly, and compute the real observed speedup, comparing it against the naive expectation of a clean 4x improvement with 4 workers. Separately, compute Amdahl's Law's predicted speedup by hand for a workload that is only 50% parallelizable, using 2, 4, and 16 workers in turn, and observe directly how quickly the returns diminish as worker count increases.

Real-world use of parallel computation

Training large machine learning models relies heavily on the naturally parallel structure of matrix operations, distributing large matrix multiplications across many GPU cores simultaneously, exactly this lecture's matrix-multiplication example at industrial scale. Web servers handle many independent client requests simultaneously, precisely this lecture's "chopping vegetables" case, since one user's request genuinely does not depend on another's. Scientific simulations, such as weather modeling or physics simulations, split a large computational grid into independent regions computed in parallel, with careful, deliberate handling required only at the boundaries where regions interact — a real, more sophisticated instance of identifying which portions of a large computation are genuinely independent.

Quiz — apply Amdahl's Law

Given a workload that is 75% parallelizable, compute the speedup achieved using 4 workers, before checking the answer, using this lecture's Amdahl's Law formula.

A second quiz check — the hard ceiling

For the same workload that is 75% parallelizable, determine the maximum possible speedup achievable with an unlimited number of workers, before checking the answer.

Second quiz — answer

The maximum possible speedup, taking the number of workers to infinity, is $1/(1 - 0.75) = 1/0.25 = 4\times$. No matter how many workers are ever added to this workload — 100, 1,000, or a million — a genuine 4x speedup can never be exceeded, since the fixed 25% sequential fraction sets this hard ceiling entirely on its own, independent of parallel hardware availability. This is worth internalizing precisely: throwing more workers at a partially sequential workload has rapidly diminishing value well before reaching this ceiling, exactly the curve drawn earlier in this lecture.

Quiz — answer

Substituting into Amdahl's Law: $\text{speedup} = 1/((1 - 0.75) + 0.75/4) = 1/(0.25 + 0.1875) = 1/0.4375 \approx 2.29\times$. This is notably less than the naive expectation of a full 4x speedup from 4 workers, despite 75% of the workload genuinely being parallelizable — exactly Amdahl's Law's central, sobering lesson: even a substantially parallelizable workload has a meaningfully bounded speedup once its fixed sequential fraction is accounted for honestly.

Summary

Some problems, such as independent map operations and merge sort's two recursive halves, parallelize naturally, because their subproblems genuinely do not depend on each other. Others, such as linked-list traversal and Bellman-Ford's relaxation rounds, are fundamentally sequential, since each step genuinely requires the previous step's completed result. Amdahl's Law formalizes the resulting speedup limit precisely: $1/((1 - p) + p/N)$, with a hard ceiling of $1/(1 - p)$ that no number of additional workers can ever exceed. Real, measured speedup is always somewhat less than the raw number of workers, due to genuine overhead, and should always be verified directly rather than assumed. Merge sort's two recursive halves parallelize naturally, while its final merge step is a concrete, already-familiar instance of Amdahl's Law's sequential fraction. Next lecture introduces randomized algorithms, which trade a worst-case guarantee for strong expected performance — precisely the shape of trade-off this entire course has built the judgment to recognize and evaluate carefully.